claude-windows / ac48091f-a4f4-4268-b968-75205d9a50ff

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ac48091f-a4f4-4268-b968-75205d9a50ff\subagents\workflows\wf_b8ec0894-3cb\agent-a5ff94826f9097703.jsonl`
- SHA-256: `4e749f51426798858f14a8ea9b61cfc5a82d5198e60681edc32bac4927a6c74e`
- Source modified: `2026-06-21T19:17:28+00:00`
- Imported at: `2026-07-08T15:59:55+00:00`
- project: `wf_b8ec0894-3cb`
- session_id: `ac48091f-a4f4-4268-b968-75205d9a50ff`

Transcript

1. user (2026-06-21T19:15:48.459Z)

```
You are a deploy-readiness SKEPTIC. Below are structured verdicts from verifiers who checked khelsutra's deploy/DEPLOY_ALPHA.md against engine origin/master + khelsutra master. The deploy itself is owner-gated (Cloudflare dashboard, cloudflared on the box, Gemini key rotation) ? the owner will follow this doc for ~an afternoon. Your job: assuming the owner does exactly what the doc says, what would STILL break, surprise, or unsafe them? Look for: ordering hazards, an unstated prerequisite, a config the engine reads from a different place than the doc implies, a default that's unsafe, a step that depends on something not yet merged, or a verified gap nobody connected. Also call out what remains UNVERIFIED (could not be determined locally ? e.g. needs a real Cloudflare account or the live box).

VERDICTS (JSON):
[
  {
    "id": "deploy-alpha-2.1-auth-extra",
    "claim": "§2.1: `pip install -e .[auth]` installs PyJWT[crypto], and that extra is REQUIRED when edge auth is on (\"else every request 500s\").",
    "verdict": "partial",
    "evidence": [
      "engine origin/master:pyproject.toml:56 ? `auth = [\"PyJWT[crypto]>=2.8\"]` declares an optional-dependency group named literally `auth` pinning PyJWT (crypto extra) >=2.8; so `pip install -e .[auth]` does install PyJWT[crypto].",
      "engine origin/master:pyproject.toml:43,55 ? `auth` is under `[project.optional-dependencies]`; comment marks it DEFAULT-OFF, only needed when `security.edge_auth_enabled=True`. PyJWT is an EXTRA, not a hard runtime dep (expected ? base runtime works without it).",
      "engine origin/master:backend/api/auth.py:71-77 ? jwt is lazy-imported; if the auth extra is absent, `verify_cf_access_token` raises EdgeAuthError('PyJWT[crypto] is required for edge auth ? install the auth extra (pip install -e .[auth])').",
      "engine origin/master:backend/main.py:571-573 ? on /api/process, `edge_auth.resolve_tenant(...)` is wrapped in `except edge_auth.EdgeAuthError -> HTTPException(status_code=401)`. So a missing-PyJWT failure surfaces as a 401, NOT a 500 ? contradicting the doc's '500s' justification.",
      "engine origin/master:backend/config/startup.py:130-137 ? boot posture only WARNs (does not fail) when edge_auth is ON and PyJWT isn't importable; matches DEPLOY_ALPHA.md:65 and also points to `pip install -e .[auth]`.",
      "khelsutra deploy/DEPLOY_ALPHA.md:43 ? doc text: 'pip install -e .[auth]  # PyJWT[crypto] ? REQUIRED when edge auth is on, else every request 500s'."
    ],
    "discrepancy": "Doc (DEPLOY_ALPHA.md:43) claims a missing `auth` extra makes \"every request 500s\". Code on origin/master returns a clean 401 instead: with edge_auth on and PyJWT absent, `verify_cf_access_token` (backend/api/auth.py:71-77) raises EdgeAuthError, which backend/main.py:572-573 maps to HTTPException(status_code=401). The core packaging claim (the `auth` extra exists and pins PyJWT[crypto]>=2.8, and is needed when edge auth is on) is correct; only the \"500s\" failure-mode wording is wrong ? it should say 401 (requests are rejected
```

```
unauthenticated, not server-errored).",
    "owner_action_note": "Without the `auth` extra, edge-auth requests are not silently broken
at 500 ? they fail closed with 401 (and the engine logs a WARN at boot per startup.py:136-137),
so the deploy is \"safe but unusable\" rather than crashing. Owner should still install
`.[auth]` on the exposed host for auth to actually pass. Minor doc fix: change \"else every
request 500s\" to \"else every request is rejected 401\".",
    "severity": "minor"
  },
  {
    "claim": "§2.2: the engine config has a `security` section with keys edge_auth_enabled,
allowed_video_root, upload_dir, cf_team_domain, cf_access_aud ? all real, with the described
meaning.",
    "id": "deploy-alpha-2.2-security-config",
    "verdict": "confirmed",
    "evidence": [
      "engine origin/master:backend/config/models.py:280 ? `class SecurityConfig(BaseModel):`
defines the section",
      "engine origin/master:backend/config/models.py:404 ? `security: SecurityConfig =
Field(default_factory=SecurityConfig)` attaches it to root AppConfig under the key `security`
(matching config.json `\"security\": {...}` in DEPLOY_ALPHA.md:50)",
      "engine origin/master:backend/config/models.py:283 ? `allowed_video_root: Optional[str] =
None` (path-jail; comment: /api/process video_path must resolve under this dir) ? matches doc
'the path-jail ? REQUIRED when exposed' (DEPLOY_ALPHA.md:52)",
      "engine origin/master:backend/config/models.py:289 ? `upload_dir: str =
\"output/uploads\"`; comment 'set allowed_video_root to contain upload_dir or /api/process will
reject the uploaded paths' ? matches doc 'MUST sit UNDER allowed_video_root'
(DEPLOY_ALPHA.md:53,127)",
      "engine origin/master:backend/config/models.py:303 ? `edge_auth_enabled: bool = False`
(DEFAULT-OFF, comment line 297) ? matches doc 'turn the CF-Access JWT verify ON' and §7 'You
MUST set edge_auth_enabled=true to expose' (DEPLOY_ALPHA.md:51,128); default is False not true",
      "engine origin/master:backend/config/models.py:304 ? `cf_team_domain: str = \"\"  # e.g.
https://<team>.cloudflareaccess.com (issuer + /certs JWKS)` ? matches doc meaning
(DEPLOY_ALPHA.md:54)",
      "engine origin/master:backend/config/models.py:305 ? `cf_access_aud: str = \"\"  # the CF
Access application AUD tag the token must carry` ? matches doc 'the CF Access application AUD'
(DEPLOY_ALPHA.md:55)",
      "engine origin/master:backend/config/startup.py:91-126 ? boot posture reads exactly these
keys (edge_auth_enabled gate, allowed_video_root jail, cf_team_domain/cf_access_aud) and fails
fast when exposed-but-misconfigured, matching DEPLOY_ALPHA.md:65"
    ],
    "discrepancy": null,
    "owner_action_note": "Doc names exactly 5 keys; SecurityConfig actually carries more
(max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[\"mp4\",\"mov\"]). Not a defect
? the doc's 5 named keys all exist with matching types/defaults/meaning, and the extras have
safe defaults the owner need not set for the alpha CUJ. Note for the owner: the engine's whole-
file upload cap default is 4 GB (max_upload_mb), independent of Cloudflare's 100 MB free-proxy
body cap called out in §7 ? the two limits are separate layers.",
    "severity": "none"
  },
  {
    "claim": "The boot posture (engine #294) fails fast if edge_auth_enabled is on but
allowed_video_root OR cf_team_domain OR cf_access_aud is missing, and warns if PyJWT isn't
installed ? so a half-configured exposure can't silently come up.",
    "id": "deploy-alpha-2.2-boot-posture",
    "verdict": "confirmed",
    "evidence": [
      "origin/master:backend/config/startup.py:97-99 ? _exposure_checks returns [] (strict no-
op) unless security.edge_auth_enabled is True (default False), so the posture only engages when
edge auth is ON",
      "origin/master:backend/config/startup.py:104-113 ? allowed_video_root unset/blank =>
StartupCheck(LEVEL_ERROR, 'EXPOSED_NO_VIDEO_ROOT', 'security.edge_auth_enabled is ON (engine
exposed) but security.allowed_video_root is unset ? an authenticated tenant could make
/api/process read an arbitrary local file. Set allowed_video_root to a jail directory before
exposing the engine.')",
      "origin/master:backend/config/startup.py:117-127 ? `if not team or not aud` (EITHER
```

missing trips it) => StartupCheck(LEVEL_ERROR, 'EXPOSED_EDGE_AUTH_INCOMPLETE', 'security.edge_auth_enabled is ON but {missing} is not configured ? the Cf-Access JWT cannot be verified (every request would fail). Set security.cf_team_domain (e.g. https://<team>.cloudflareaccess.com) and security.cf_access_aud (the CF Access application AUD tag).')",
      "origin/master:backend/config/startup.py:131-138 ? PyJWT not importable => StartupCheck(LEVEL_WARN, 'EXPOSED_AUTH_EXTRA_MISSING', 'security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ? Cf-Access verification will fail at request time. Install the auth extra (pip install -e .[auth]).') ? WARN, not ERROR",
      "origin/master:backend/config/startup.py:269-281 ? enforce_startup_checks logs each finding and `if errors and raise_on_error: raise StartupCheckError(errors)`; only LEVEL_ERROR entries are 'errors', so the PyJWT WARN never raises",
      "origin/master:backend/main.py:854-857 ? _enforce_startup_or_exit calls enforce_startup_checks(config, logger=logger, include_exposure=True); on StartupCheckError it logs '[STARTUP] refusing to start' and sys.exit(1) (fail fast = process exits 1)",
      "origin/master:backend/main.py:940 ? _enforce_startup_or_exit(global_config) runs inside main() before the --server/--app branch (line ~1061), so a `python -m backend.main --server` boot hits it; the exposure check is the ONLY caller passing include_exposure=True",
      "origin/master:backend/worker/__main__.py:111 ? the worker calls enforce_startup_checks WITHOUT include_exposure (defaults False), so the exposure posture is genuinely main()/server-side only, matching the doc's framing"
    ],
    "discrepancy": "No substantive discrepancy. One precision nuance, not a doc error: the exposure enforcement at backend/main.py:940 executes in main() on EVERY invocation (it always passes include_exposure=True), including a CLI `--video-path` single-shot run ? it is not literally gated on the --server flag. What makes it a no-op for non-exposed use is edge_auth_enabled=False, NOT the absence of --server. Since the alpha deploy in §2.3 runs `python -m backend.main --server` with edge_auth_enabled=true, the claim's behavior holds exactly as written.",
    "owner_action_note": "The fatal errors trip ONLY when edge_auth_enabled=true (which §2.2's config.json sets) ? confirmed correct for this runbook. The owner should recognize two exact ERROR codes/messages on a bad boot and the process exiting with code 1 and logging '[STARTUP] refusing to start': 'EXPOSED_NO_VIDEO_ROOT' (allowed_video_root missing) and 'EXPOSED_EDGE_AUTH_INCOMPLETE' (cf_team_domain and/or cf_access_aud missing), plus a non-fatal WARN 'EXPOSED_AUTH_EXTRA_MISSING' if PyJWT isn't installed. Note: PyJWT-missing is only a WARN, so the engine WILL still boot without the auth extra and then 500 every authenticated request at verify time ? §2.1's `pip install -e .[auth]` is what actually prevents that; the boot posture does not enforce it. Also note a 4th finding not mentioned in the doc: if RALLY_ALLOWED_HOSTS is unset the boot emits a WARN 'EXPOSED_HOST_ALLOWLIST_LOOPBACK' (the Host guard defaults loopback-only and would 400 forwarded requests) ? harmless to the claim but the owner may see it.",
    "severity": "none"
  },
  {
    "id": "deploy-alpha-2.3-bind",
    "claim": "§2.3: `python -m backend.main --server` runs the engine and binds 127.0.0.1:8000 (the safe default ? \"do NOT change the bind\"); PR #296 (P0c) added a config/profile-driven bind + DNS-rebinding Host-header guard.",
    "verdict": "partial",
    "evidence": [
      "origin/master:backend/main.py:866 ? parser.add_argument('--server', action='store_true', ...): `python -m backend.main --server` IS a valid invocation (sub-claim a CONFIRMED). __main__ dispatches via `import backend.main as _bm; _bm.main()`.",
      "origin/master:backend/main.py:870 ? --port default=8000; main.py:1081 `port = args.port` for the --server branch ? default port 8000 CONFIRMED.",
      "origin/master:backend/main.py:115-128 ? _resolve_bind_host() precedence: --host > RALLY_BIND_HOST env > 0.0.0.0 WHEN security.edge_auth_enabled is True > 127.0.0.1 default. 127.0.0.1 is the default ONLY when edge_auth is OFF and neither override set.",
      "origin/master:backend/main.py:1080 ? the --server branch calls bind_host = _resolve_bind_host(global_config, args.host); with no --host/RALLY_BIND_HOST the bind is config-driven. main.py:1083-1087 prints a ? warning and binds the resolved host when it is not loopback.",
      "origin/master:backend/config/models.py:303 ? edge_auth_enabled: bool = False (default OFF); models.py:404 security: SecurityConfig.",
      "origin/master:backend/config/loader.py:120-200 + main.py:947 `global_config = load_app_config()` ? config.json merges into the typed AppConfig, so the §2.2 config.json

`security.edge_auth_enabled: true` sets global_config.security.edge_auth_enabled=True ? _resolve_bind_host returns 0.0.0.0, NOT 127.0.0.1.",
      "origin/master:backend/main.py:88-111 ? _allowed_hosts_from_env() + TrustedHostMiddleware (DNS-rebinding Host-header guard) is present and ON by default; default allowlist=['localhost','127.0.0.1','testserver'], override via RALLY_ALLOWED_HOSTS (sub-claim re: #296 guard CONFIRMED).",
      "origin/master:backend/config/startup.py:91-147 ? exposure-safety boot posture (PR #294) gated on edge_auth_enabled; startup.py:146 specifically: 'edge_auth_enabled is ON but RALLY_ALLOWED_HOSTS is unset ? the Host-header guard...' confirming that exposing requires setting RALLY_ALLOWED_HOSTS (the loopback-only default 400s forwarded requests)."
    ],
    "discrepancy": "Doc says following §2.2 then §2.3 binds 127.0.0.1:8000; CODE binds 0.0.0.0:8000. The §2.2 config the doc instructs the owner to create sets `security.edge_auth_enabled: true`. _resolve_bind_host (main.py:115-128) returns 0.0.0.0 ? not 127.0.0.1 ? exactly when edge_auth_enabled is true (and no --host/RALLY_BIND_HOST override). So the §2.3 inline comment 'binds 127.0.0.1:8000 (the safe default ? do NOT change the bind)' is FALSE for the very config this runbook prescribes: with edge_auth on, --server binds ALL interfaces. (The bind is then SAFE because it sits behind the cloudflared outbound tunnel + the in-app Cf-Access JWT verify + the Host-header guard ? but the doc's literal 127.0.0.1 claim and 'no inbound port' §0/§6 framing are technically wrong: the socket is 0.0.0.0-bound; what actually prevents inbound exposure is that no inbound firewall port is opened and cloudflared dials out, NOT the loopback bind the doc relies on.)",
    "owner_action_note": "Two real things the owner must know/do beyond the doc text: (1) With the §2.2 config (edge_auth_enabled:true), the engine binds 0.0.0.0 ? the §6 posture test `curl http://<box-public-ip>:8000/api/health ? refused/timeout` will NOT necessarily hold from the socket level; it only holds if no inbound firewall port is open on the box. The owner MUST ensure no inbound port 8000 is opened in the box/cloud firewall (the cloudflared tunnel is outbound-only and does not require it). (2) When exposed (edge_auth on ? 0.0.0.0), the Host-header guard defaults to loopback-only (['localhost','127.0.0.1','testserver']); a forwarded request arriving with Host: api.khelsutra.guru will be 400'd by TrustedHostMiddleware unless RALLY_ALLOWED_HOSTS is set to the public host (or '*' behind the gate). The boot posture (startup.py:146) WARNS about this but the §2.2 config block in the runbook does NOT instruct setting RALLY_ALLOWED_HOSTS, and the cloudflared ingress sends Host: api.khelsutra.guru by default ? so as written, every tunneled /api request would 400 until RALLY_ALLOWED_HOSTS is set. To force the doc's literal 127.0.0.1 bind while still tunneling, the owner could set RALLY_BIND_HOST=127.0.0.1 (cloudflared dials 127.0.0.1:8000 anyway), which is arguably the more defensible posture and matches the doc's intent.",
    "severity": "blocker"
  },
  {
    "id": "deploy-alpha-cors-config",
    "claim": "Engine locks CORS to the SPA origin via env RALLY_CORS_ALLOW_ORIGINS (no code change), with allow_credentials=false, and answers the cross-origin OPTIONS preflight; doc also tells the owner to \"create/edit config.json on the box\".",
    "verdict": "confirmed",
    "evidence": [
      "origin/master backend/main.py:65-72 - _cors_origins_from_env() reads RALLY_CORS_ALLOW_ORIGINS from os.environ (comma-split, default ['*']); docstring says read from ENV not config.json so import does zero filesystem I/O. Confirms env-driven, no engine code change.",
      "origin/master backend/main.py:79-84 - app.add_middleware(CORSMiddleware, allow_origins=_cors_origins_from_env(), allow_credentials=False, allow_methods=['*'], allow_headers=['*']). Confirms allow_credentials=False and the origin lock. Wired at module import (line 79), independent of any config load.",
      "FastAPI/Starlette CORSMiddleware answers the cross-origin OPTIONS preflight automatically for an allowed origin (standard middleware; the engine adds no custom preflight handler). origin/master tests/test_edge_auth.py:205-211 (test_cors_origins_from_env) asserts {}->['*'], ''->['*'], 'https://a, https://b'->two-origin list - locks the env-parsing contract.",
      "origin/master backend/utils/app_home.py:36-56 - app_home() returns Path(RALLY_APP_HOME) when set else Path('.') (CWD); default_config_path() returns app_home()/'config.json'. So config.json discovery = <RALLY_APP_HOME>/config.json when env set, else ./config.json (CWD-relative).",
      "origin/master backend/config/loader.py:108-120 - load_config(path=None) resolves cfg_path = default_config_path() when path is None (RALLY_APP_HOME-aware). Missing/unreadable file -> defaulted AppConfig (non-fatal).",

```
      "origin/master backend/main.py:860,939-940,1063-1092 - server boot main() calls
load_app_config() with NO arg (line 939, env-aware) then _enforce_startup_or_exit(config) then
uvicorn.run(app) under --server/--app. So the EXPOSED server reads config from the resolved app-
home path, and the M9/exposure startup checks run against THAT config.",
      "origin/master scripts/doctor.py:7-9,52-69 - --setup WRITES config.json via
default_config_path() (RALLY_APP_HOME else CWD), the SAME path the server READS; comment
explicitly notes this fixed the prior P0a bug where doctor wrote repo-root while the server read
CWD."
    ],
    "discrepancy": "Not a CORS discrepancy - part (a) is fully correct. The config.json wrinkle:
the doc (§2.2) says \"Create / edit config.json on the box\" without saying WHERE. The real
discovery path is RALLY_APP_HOME-governed: when RALLY_APP_HOME is set, the engine reads (and
--setup writes) <RALLY_APP_HOME>/config.json; when unset, it is ./config.json relative to the
server's CWD (NOT the repo root). The owner CAN put config.json in the wrong place: if they edit
./config.json but launch the server from a different CWD (or with RALLY_APP_HOME exported), the
engine silently reads a different file (missing file -> all defaults, edge_auth_enabled=False ->
no auth, default CORS '*'). Note: doc §2.2 sets edge_auth/jail in config.json but exports
RALLY_CORS_ALLOW_ORIGINS as an env var, so CORS is unaffected by the config.json-location
pitfall; only the security/jail/AUD block (which lives in config.json) is at risk of being
silently ignored.",
    "owner_action_note": "config.json resolution order on origin/master: (1) explicit path arg
to load_config (not used by the server boot path); else (2) $RALLY_APP_HOME/config.json when
RALLY_APP_HOME is set (~ expanded); else (3) ./config.json relative to the process CWD. Safe
recipe: either (a) run with RALLY_APP_HOME UNSET and start the server with CWD = the directory
that holds config.json, OR (b) export RALLY_APP_HOME=<dir> and run `python -m backend.main
--setup` (writes <dir>/config.json) then edit THAT file - --setup and the server then agree by
construction. Verify after editing: the security block actually took effect by confirming the
engine fails-fast (PR #294) if edge_auth_enabled=true but the jail/AUD are missing - if it boots
silently with auth claimed on, the engine read a DIFFERENT (or absent) config.json and is
running on defaults (edge_auth OFF). CORS itself is env-driven (RALLY_CORS_ALLOW_ORIGINS) so it
is immune to the config.json-location pitfall; ALSO requires the separate Cloudflare Access
\"Bypass OPTIONS to CORS preflight\" toggle (doc §4.5 step 5) or the preflight is redirected to
login before the engine's CORS middleware ever answers it.",
    "severity": "minor"
  },
  {
    "id": "deploy-alpha-allowed-video-root-jail",
    "claim": "§2.2/§7: allowed_video_root is a path-jail REQUIRED when exposed; /api/process
rejects any video path outside it; upload_dir MUST sit under allowed_video_root or uploads are
rejected.",
    "verdict": "partial",
    "evidence": [
      "origin/master:backend/utils/paths.py:53-64 ? validate_input_path(path, allowed_root):
always rejects null bytes; when allowed_root is set, delegates to resolve_under_root(path,
allowed_root).",
      "origin/master:backend/utils/paths.py:39-50 ? resolve_under_root:
rp=os.path.realpath(path); rr=os.path.realpath(root); contained =
os.path.commonpath([rp,rr])==rr; else raises ValueError 'escapes the allowed root'. realpath
collapses '..' and resolves symlinks BEFORE the commonpath test, and the except-ValueError
(cross-drive) branch marks non-contained ? so traversal, symlink, and absolute-path escapes are
all caught.",
      "origin/master:backend/main.py:575-583 ? /api/process reads allowed_root from
security.allowed_video_root, calls validate_input_path FIRST then storage.resolve_input_ref;
ValueError -> HTTPException(400). An out-of-jail path is a clean 400 before any work.",
      "origin/master:tests/test_path_safety.py:33-50 ? pins containment (inside ok / outside
raises) AND traversal: resolve_under_root(root/'..'/escape.mp4, root) raises ValueError.",
      "origin/master:backend/api/uploads.py:124-145 ? upload_complete writes
final_path=os.path.join(upload_dir, filename) and returns {'path': os.path.abspath(final_path)}
for the SPA to POST to /api/process; the uploaded file's path is validated by the SAME jail at
process time, so if upload_dir is not under allowed_video_root the upload 400s. Confirms the
upload_dir-under-jail requirement is real.",
      "origin/master:backend/config/models.py:283-289 ? SecurityConfig: allowed_video_root
default None; upload_dir default 'output/uploads'; comment 287-288: 'In hosted mode, set
allowed_video_root to contain upload_dir or /api/process will reject the uploaded paths.'",
      "origin/master:backend/config/startup.py:90-113 + backend/main.py:845-857,940 ? when
```

security.edge_auth_enabled is ON, _exposure_checks emits LEVEL_ERROR EXPOSED_NO_VIDEO_ROOT if allowed_video_root is unset; the server's _enforce_startup_or_exit raises StartupCheckError -> sys.exit(1). So the jail is genuinely REQUIRED-to-boot when exposed (not just advisory).",
      "origin/master:tests/test_api_endpoints.py:609-622 ? test_process_hosted_mode_rejects_nonlocal_uri: with allowed_video_root set, POST /api/process with 'gs://b/x.mp4', 's3://b/x.mp4', AND 'gdrive://Sharing/x.mp4' each returns 400; a local file UNDER the root returns 202. This is the gs:// finding.",
      "origin/master:backend/main.py:584-588 ? engine's own comment: 'a configured allowed_video_root (hosted mode) rejects a non-local URI as out-of-root.'"
    ],
    "discrepancy": "DOC INTERNAL CONTRADICTION on gs:// under the exposed/jailed config. The doc's CUJ (§6) tells testers to \"paste a gs://? video (or a path under allowed_video_root)\" and §7 says to \"pre-stage to a bucket and ingest the gs:// URI (already supported)\" ? but §2.2 REQUIRES allowed_video_root to be set to expose. CODE-SAYS (tests/test_api_endpoints.py:609 + paths.py:39-50): when allowed_video_root is set, validate_input_path runs BEFORE resolve_input_ref and os.path.realpath('gs://bucket/x.mp4') resolves to a LOCAL relative path that escapes the jail -> ValueError -> 400. So every gs://, s3://, gdrive:// URI is REJECTED on exactly the exposed config the doc instructs the owner to deploy. gs:// does NOT 'bypass the jail by design'; it is blocked by it. The two doc instructions (use allowed_video_root AND paste gs://) are mutually exclusive on the alpha box. The §7 'already supported' clause is true ONLY when allowed_video_root is unset (single-user/local), which is the opposite of the exposed posture this runbook ships.",
    "owner_action_note": "The path-jail mechanics in §2.2/§7 are fully correct and well-tested (traversal/symlink/absolute-path resistant; upload_dir-under-jail enforced at /api/process; jail required-to-boot when edge_auth_enabled). BUT do not promise gs:// ingest in the alpha CUJ as written: with allowed_video_root set, the engine 400s every gs://, s3://, and gdrive:// URI (tests/test_api_endpoints.py:609). To actually support pasting a gs:// URI for a tester, the doc/owner needs ONE of: (a) drop gs:// from the §6 CUJ and §7 'already supported' line for the exposed box and rely on browser upload into upload_dir-under-jail; or (b) get an engine change that exempts recognized storage schemes (gs://, s3://, gdrive://) from the local path-jail (e.g. scheme-check before resolve_under_root, or validate AFTER resolve_input_ref and jail only local refs) ? that is engine work, not a config flip, and must be done carefully so it doesn't reopen the arbitrary-local-file hole. As shipped, the safe alpha flow is: upload (kept inside the jail) ? NOT gs://.",
    "severity": "blocker"
  },
  {
    "claim": "§0/§2.2: the engine ALSO verifies the Cf-Access JWT in-app (defence in depth, gated by security.edge_auth_enabled), so a direct tunnel hit can't spoof identity.",
    "id": "deploy-alpha-edge-auth-inapp-jwt",
    "verdict": "confirmed",
    "evidence": [
      "ENGINE origin/master backend/api/auth.py:verify_cf_access_token ? does REAL RS256 verification: jwt.decode(token, signing_key.key, algorithms=[\"RS256\"], audience=aud, issuer=_issuer(team_domain), options={\"require\": [\"exp\",\"iss\",\"aud\"]}); raises EdgeAuthError on bad sig/exp/aud/iss. NOT a mere header-presence check.",
      "ENGINE origin/master backend/api/auth.py:_get_jwks + _default_jwks_fetcher ? fetches JWKS from cf_team_domain + /cdn-cgi/access/certs, validates shape ({\"keys\" in data} else EdgeAuthError), module-level cache keyed by team_domain with _JWKS_TTL_SEC=600 (re-fetch ? every 10 min). signing key selected by token's kid via PyJWKSet.",
      "ENGINE origin/master backend/api/auth.py:resolve_tenant ? strict NO-OP when edge_auth_enabled is False/absent (returns None = local single-user); when enabled, requires the Cf-Access-Jwt-Assertion header (case-insensitive) and raises EdgeAuthError if missing; also raises if enabled-but-cf_team_domain/cf_access_aud unset (incl. whitespace-only).",
      "ENGINE origin/master backend/main.py:567-573 ? call site: owner_id = edge_auth.resolve_tenant(request.headers, global_config); except EdgeAuthError -> raise HTTPException(status_code=401). So a missing/invalid/spoofed token yields 401 before any work, and tags job owner_id on success.",
      "ENGINE origin/master backend/config/models.py:303-305 ? SecurityConfig.edge_auth_enabled: bool = False (DEFAULT-OFF), cf_team_domain: str = \"\", cf_access_aud: str = \"\"; comment confirms these are the public JWKS issuer + AUD, no secrets.",
      "ENGINE origin/master tests/test_edge_auth.py ? proves real verification: rejects expired (exp_delta:-10), wrong aud, wrong iss, bad signature (different key, same kid), alg:none bypass (test_alg_none_token_rejected), and HS256 algorithm-confusion (test_hs256_algorithm_confusion_rejected). NO-OP-when-OFF tests: resolve_tenant off returns None

```
even with a token; missing-header->401 (test_process_edge_auth_missing_token_401); whitespace-
only config raises.",
      "khelsutra deploy/DEPLOY_ALPHA.md:22 ? the §0 claim text; line 43 already warns 'pip
install -e .[auth]  # PyJWT[crypto] ? REQUIRED when edge auth is on, else every request 500s';
lines 51-55 set edge_auth_enabled/cf_team_domain/cf_access_aud; line 65 notes PR #294 boot
posture fails fast if cf_team_domain/cf_access_aud missing."
    ],
    "discrepancy": null,
    "owner_action_note": "Implementation is strong ? no weaknesses found. alg is pinned to RS256
(alg:none AND HS256-confusion are both explicitly tested and rejected), aud + iss + exp are
required and verified, JWKS is fetched from the team domain and cached with a 600s TTL. Two
operational notes the owner must honor (both already in the doc): (1) the in-app verify needs
PyJWT[crypto] ? install `pip install -e .[auth]`, else verify_cf_access_token raises
EdgeAuthError and every request 401s/500s when edge_auth_enabled=true (doc line 43 covers this).
(2) The in-app JWT verify is wired only at the /api/process submit path (backend/main.py:571);
it is the work-submitting route, so for alpha it is the load-bearing one, but it is NOT a global
FastAPI dependency across every endpoint ? the Cloudflare Access edge gate (in front of the
tunnel) remains the primary auth boundary; the in-app check is the defence-in-depth backstop the
doc claims it is.",
    "severity": "none"
  },
  {
    "id": "deploy-alpha-spa-vite-api-base",
    "claim": "SPA built with VITE_API_BASE=https://api.khelsutra.guru/api (PR #64) defaulting to
same-origin /api; Pages build root=web, `npm run build`, output web/dist; browser chunks large
uploads to <=90MB (under the 100MB CF cap); reel downloads cross-origin via Content-
Disposition.",
    "verdict": "partial",
    "evidence": [
      "web/src/api/client.ts:23-24 ? apiBase() returns (import.meta.env.VITE_API_BASE ??
'/api').replace(/\\/+$/, '') ? reads VITE_API_BASE, falls back to same-origin /api, trims
trailing slash. BASE consumed at line 27.",
      "web/src/vite-env.d.ts:9 ? readonly VITE_API_BASE?: string (typed env var).",
      "web/src/api/client.test.ts:27-37 ? tests assert default '/api' when unset AND
VITE_API_BASE='https://api.khelsutra.guru/api' for the split deploy (plus trailing-slash
trim).",
      "web/package.json:9 ? 'build': 'vite build'. No build.outDir override in
web/vite.config.ts, so Vite's default output dir 'dist' applies relative to root=web ->
web/dist. Matches doc §3 (DEPLOY_ALPHA.md:81-83).",
      "web/src/api/client.ts:218-219 ? partMb derived from engine's init.max_part_mb (NOT a
hardcoded 90), fallback 95 when missing/NaN; partBytes = partMb*1024*1024. Upload entry:
web/src/hooks/useUpload.ts:52 -> uploadFile() client.ts:210-239 -> sequential uploadPart PUTs
client.ts:155-184.",
      "ENGINE origin/master:backend/config/models.py:293 ? max_part_mb: int = 95; and
origin/master:backend/api/uploads.py:49 ? getattr(sec, 'max_part_mb', 95) or 95. Default chunk =
95 MB, not <=90 MB (still under the 100 MB CF cap, so it functions).",
      "web/src/components/CompileResult.tsx:68-74 ? <a href={build.url}
download={build.filename}> where build.url=highlightUrl() (client.ts:127-129) on the cross-
origin api. host; doc §7 line 129 correctly notes cross-origin <a download> hints are ignored so
the header is authoritative.",
      "ENGINE origin/master:backend/main.py:690 ? download_highlight returns FileResponse(path,
media_type=media, filename=name); Starlette FileResponse with a filename kwarg emits Content-
Disposition: attachment; filename=..., confirming the cross-origin download relies on Content-
Disposition."
    ],
    "discrepancy": "Doc §7 (DEPLOY_ALPHA.md:130) says large matches \"upload as <=90 MB
chunks\". Code says 95 MB: the SPA does not hardcode a 90 MB chunk size ? it honors the engine's
max_part_mb from /upload/init (web/src/api/client.ts:218), whose default is 95 (engine
origin/master backend/config/models.py:293 and backend/api/uploads.py:49). So with default
config chunks are 95 MB, not <=90 MB. 95 MB is still under Cloudflare's 100 MB cap so it works,
but the specific \"<=90 MB\" number in the doc is inaccurate (chunk size is engine-configurable,
defaulting to 95).",
    "owner_action_note": "If the owner wants the actual chunk size at/below the doc's stated 90
MB margin for the 100 MB CF cap, set security.max_part_mb to <=90 in the engine config.json
(§2.2); otherwise update the doc to say ~95 MB (or \"engine-configured, default 95 MB\"). The 95
```

```
MB default already fits under the 100 MB cap, so the deploy is functionally safe either way.
Note vite.config.ts has no explicit build.outDir, so the web/dist output relies on Vite's
default ? correct today, but a future outDir change would silently break the Pages \"Build
output directory: web/dist\" setting.",
    "severity": "minor"
  },
  {
    "id": "deploy-alpha-s5-cloudflared-config",
    "claim": "deploy/cloudflared.config.example.yml defines the ingress (only api.khelsutra.guru
? http://127.0.0.1:8000) and is correct/complete for `cloudflared tunnel run`.",
    "verdict": "confirmed",
    "evidence": [
      "/c/Users/avidu/Projects/khelsutra/deploy/cloudflared.config.example.yml:11 ? `tunnel:
<TUNNEL_UUID>` placeholder present",
      "/c/Users/avidu/Projects/khelsutra/deploy/cloudflared.config.example.yml:12 ?
`credentials-file: /etc/cloudflared/<TUNNEL_UUID>.json` placeholder present",
      "/c/Users/avidu/Projects/khelsutra/deploy/cloudflared.config.example.yml:17-18 ? single
ingress rule `- hostname: api.khelsutra.guru` ? `service: http://127.0.0.1:8000` (exact match to
§5, the only hostname rule)",
      "/c/Users/avidu/Projects/khelsutra/deploy/cloudflared.config.example.yml:26 ? `- service:
http_status:404` is the final catch-all (cloudflared requires this; it IS last)",
      "/c/Users/avidu/Projects/khelsutra/deploy/cloudflared.config.example.yml:19-23 ? well-
formed `originRequest` (connectTimeout: 30s, noTLSVerify: false) ? valid cloudflared keys, no
validation breakage",
      "/c/Users/avidu/Projects/khelsutra/deploy/DEPLOY_ALPHA.md:106 ? §5 instructs copy to
`/etc/cloudflared/config.yml`, matching the config header's documented path (lines 3-4) and
cloudflared's default lookup location",
      "/c/Users/avidu/Projects/khelsutra/deploy/DEPLOY_ALPHA.md:105,108,110 ? §5 uses tunnel
name `khelsutra-studio` for create/run, while the config keys on `<TUNNEL_UUID>`; both resolve
to the same tunnel so no contradiction"
    ],
    "discrepancy": "None material. Nuance: §5's CLI commands (create/route/run) use the tunnel
NAME `khelsutra-studio`, whereas the config file keys the same tunnel by `<TUNNEL_UUID>` (lines
11-12). These are not contradictory ? the UUID is the name's identity, and §5 line 105
explicitly tells the operator to note the UUID from `tunnel create` and fill it in. The doc's §5
summary line (DEPLOY_ALPHA.md:110) accurately describes the file's ingress.",
    "owner_action_note": "Two manual steps the config file's placeholders imply but neither §5
nor the file spells out fully: (1) replace BOTH `<TUNNEL_UUID>` occurrences (tunnel: and
credentials-file:) with the real UUID from `cloudflared tunnel create khelsutra-studio`. (2)
`cloudflared tunnel create` writes the credentials JSON to `~/.cloudflared/<UUID>.json` by
default, but the config's `credentials-file` points at `/etc/cloudflared/<UUID>.json` ? the
operator must move/copy that JSON to `/etc/cloudflared/` (or run cloudflared as a user whose
`~/.cloudflared` matches) or `tunnel run` will fail to find credentials. Also ensure the process
running cloudflared can read `/etc/cloudflared/config.yml` and the credentials file
(root/service perms).",
    "severity": "none"
  },
  {
    "id": "doc-freshness-A3-A6-A7",
    "claim": "Session memory/handoff lists A3 (#74) and A6 (#75) as still-pending UX items;
verify they are merged on khelsutra master and report the §7 tracker checkbox state for A3/A6/A7
and other A-items, especially whether A7 (recording diagram) is done or open.",
    "verdict": "confirmed",
    "evidence": [
      "khelsutra git log master: A3 merged ? commit 1cd84af 'feat(site): A3 ? localize
recording-education + marketing to kn/hi, Noto on the site (#74)'; A6 merged ? commit aa8d5e9
'feat(site): A6 ? plain-language Request-Access GPU/storage questions (#75)'",
      "docs/FRIENDLY_AND_MULTILINGUAL.md:124 ? section heading '## 7. Deliverables & progress
tracker ? source of truth' (the UX/l10n tracker table)",
      "docs/FRIENDLY_AND_MULTILINGUAL.md:139 ? A3 row is ? Done, PR #74;
docs/FRIENDLY_AND_MULTILINGUAL.md:135 ? A6 row is ? Done, PR #75 (tracker rows already reflect
merged reality)",
      "docs/FRIENDLY_AND_MULTILINGUAL.md:136 ? A7 'Fix English-source recording gaps (add a
placement diagram) before localizing' is ? Todo, PR cell '?' ? A7 is STILL OPEN (no A7 commit
exists on any branch in git log)",
```

```
      "docs/FRIENDLY_AND_MULTILINGUAL.md:132-144 ? full A-item checkbox state: A1 ?(#60), A2
?(?), A3 ?(#74), A4 ?(#61), A5 ?(#62), A6 ?(#75), A7 ?(?), A8 ?(#63), A9 ?(#68); no A10+",
      "docs/FRIENDLY_AND_MULTILINGUAL.md:147 ? milestone PROSE lists 'A6 plain-language access
form' among items 'Still open for the full §9 DoD', contradicting the ? A6 row at line 135 (#75
merged)"
    ],
    "discrepancy": "Doc-internal staleness: the §7 milestone PROSE at line 147 still lists A6
('A6 plain-language access form') as 'Still open for the full §9 DoD', but the A6 tracker ROW at
line 135 is ? Done with PR #75 (commit aa8d5e9) merged on master. The tracker rows themselves
are accurate (A3 ? #74, A6 ? #75, A7 ? open). The session memory/handoff premise ? that A3 and
A6 are 'still-pending' ? is itself stale vs git master: both are merged. A7 (recording placement
diagram, line 136) and A2 (WhatsApp share, line 134) are the only genuinely open A-items.",
    "owner_action_note": "A7 (recording placement diagram) is genuinely still open (?, no PR, no
commit on any branch) ? not done. If the deploy runbook or memory implies A7 is closed, that is
wrong. Two cheap doc fixes for honesty (non-blocking): (1) drop 'A6 plain-language access form'
from the 'Still open' prose at FRIENDLY_AND_MULTILINGUAL.md:147 since A6/#75 merged; (2) refresh
session memory/handoff to mark A3/#74 and A6/#75 as merged (only A2 and A7 remain open A-items).
None of this affects the deploy itself.",
    "severity": "minor"
  }
]
```


Return a prioritized readiness assessment.

2. assistant (2026-06-21T19:15:51.737Z)

I'll verify the highest-severity claims against the actual code before assembling my assessment. Let me start by pulling the latest and reading the key files.

3. user (2026-06-21T19:15:55.391Z)

```
aa8d5e9 feat(site): A6 ? plain-language Request-Access GPU/storage questions (#75)
---BRANCH---
master
```

4. user (2026-06-21T19:15:55.419Z)

```
1       # Deploy KhelSutra Studio for alpha ? a shareable page, no SSH
2
3       > **Status:** `READY` (owner executes the cloud steps) · **Topology:** split ? SPA on
Cloudflare Pages + engine on an `api.*` cloudflared tunnel, Cloudflare Access over both (the
engine's `HOSTING_PLAN.md` §2 alpha plan). · **Cost:** ~$0/mo standing (Pages + Tunnel + Access
are free ?50 users) + Gemini per-run (~$0.05/match).
4
5       This turns the local-only flow (run engine + SPA on your box, drive it yourself) into a
**page you can share with alpha testers** ? they open `app.khelsutra.guru`, sign in (email
allowlist), point at a video, and get a reel. No SSH, no home IP exposed, no inbound ports.
6
7       ## 0. The shape
8
9       ```
10       tester browser
11          ? https
12          ?
13       Cloudflare edge (free): DNS · TLS · Access (email-OTP allowlist) in front of BOTH
hostnames
14          ??? app.khelsutra.guru  ? Cloudflare Pages         (the web/ SPA; built with
VITE_API_BASE)
15          ??? api.khelsutra.guru  ? cloudflared Tunnel (outbound-only) ?? 127.0.0.1:8000 on
YOUR box
16                                                                          ?  FastAPI engine
17                                                                          ?  (Gemini
segmenter; GPU optional)
18                                                                          ?  async jobs,
```

SQLite, output/ on disk
19   ```
20
21   - **No inbound ports.** `cloudflared` dials out; the engine stays bound to
`127.0.0.1:8000`. The box's home/public IP is never exposed.
22   - **Auth at the edge.** Cloudflare Access gates both hostnames; the engine *also*
verifies the `Cf-Access` JWT in-app (defence in depth ? `security.edge_auth_enabled`), so a
direct hit to the tunnel can't spoof identity.
23   - **Cross-origin, on purpose.** `app.` (SPA) and `api.` (engine) are different origins ?
the SPA is built with `VITE_API_BASE=https://api.khelsutra.guru/api` (PR #64) and the engine
locks CORS to the SPA origin via `RALLY_CORS_ALLOW_ORIGINS` (no engine code change ? it's
already env-driven).
24
25   > **Alternative (single-origin):** if you'd rather run **one box** that serves the SPA
and the API behind one origin (Caddy reverse-proxy, no Pages, no cross-origin), see
`LOCAL_APPLIANCE_ARCHITECTURE.md` D1. This runbook ships the **split** topology you chose
(stable Pages-hosted page + a thin API tunnel).
26
27   ---
28
29   ## 1. Prerequisites
30
31   - A **box** to run the engine: the CPU droplet `168.144.126.176` (Gemini-only ? no GPU;
fine for alpha) **or** your Windows+WSL GPU box (real WASB). `ffmpeg` + Python 3.8+ installed;
the `badminton-highlight-indexer` engine checked out.
32   - A **Cloudflare account** with the `khelsutra.guru` zone (you already run the marketing
site there).
33   - `cloudflared` installed on the box (`https://developers.cloudflare.com/cloudflare-
one/connections/connect-networks/downloads/`).
34   - A **fresh** `GEMINI_API_KEY` (rotate the chat-shared one ? it uploads downsized chunks
to Google; this is a paid + cloud action).
35
36   ---
37
38   ## 2. Engine (the `api.` side)
39
40   ### 2.1 Install (with the auth extra)
41   ```bash
42   cd badminton-highlight-indexer
43   pip install -e .[auth]   # PyJWT[crypto] ? REQUIRED when edge auth is on, else
every request 500s
44   ```
45
46   ### 2.2 The safe-exposed config
47   Create / edit `config.json` on the box:
48   ```jsonc
49   {
50   "security": {
51   "edge_auth_enabled": true,   // turn the CF-Access JWT verify ON
52   "allowed_video_root": "/srv/khelsutra/incoming", // the path-jail ? REQUIRED when
exposed
53   "upload_dir": "/srv/khelsutra/incoming/uploads", // MUST sit UNDER
allowed_video_root (see §7)
54   "cf_team_domain": "https://<your-team>.cloudflareaccess.com",
55   "cf_access_aud": "<the CF Access application AUD>" // filled in §4
56   }
57   }
58   ```
59   And export the env (a `.env` or your shell):
60   ```bash
61   export GEMINI_API_KEY="<your-rotated-key>"
62   export RALLY_CORS_ALLOW_ORIGINS="https://app.khelsutra.guru"   # lock CORS to the SPA
origin
63   ```
64

> **The engine refuses to start unsafe.** The boot posture (engine PR #294) **fails fast** if `edge_auth_enabled` is on but the `allowed_video_root` jail or `cf_team_domain`/`cf_access_aud` is missing, and **warns** if `PyJWT` isn't installed. So a half-configured exposure can't silently come up ? fix what it prints and restart.

2.3 Run

```bash
python -m backend.main --server     # binds 127.0.0.1:8000 (the safe default ? do NOT change the bind)
```

Keep it alive with a process manager (`systemd` on the droplet, mirroring `site/scripts/provision-vm.sh`'s unit; or `pm2`/`nssm` on Windows).

---

3. The SPA on Cloudflare Pages (the `app.` side)

The SPA ships as a **static Pages site**, git-connected like the marketing site (auto-deploy on merge to `master`).

1. **Cloudflare dashboard ? Workers & Pages ? Create ? Pages ? Connect to Git** ? pick the `khelsutra` repo.
2. **Build settings:**
   - **Root directory:** `web`
   - **Build command:** `npm run build`
   - **Build output directory:** `web/dist`
   - **Environment variable:** `VITE_API_BASE = https://api.khelsutra.guru/api` ? this is what points the SPA at the engine tunnel (PR #64).
3. **Custom domain:** add `app.khelsutra.guru` to the Pages project.

> Every push to `master` now rebuilds + redeploys the SPA. To preview a change first, Pages builds a per-PR preview URL.

---

4. Cloudflare Access (gate BOTH hostnames)

1. **Zero Trust ? Access ? Applications ? Add ? Self-hosted.**
2. **Application domain:** add **both** `app.khelsutra.guru` **and** `api.khelsutra.guru` to the *same* application (one app ? one shared session cookie, so a tester signs in once and both the SPA and its API calls are authorized).
3. **Policy:** Action *Allow*, rule *Emails* ? your alpha testers' addresses (free ? 50 users ? fits 10?25 testers). Use one-time-PIN (email OTP) so testers need no Cloudflare account.
4. **Copy the application `AUD` tag** ? paste into the engine's `config.json` `security.cf_access_aud` (§2.2), and set `cf_team_domain` to `https://<your-team>.cloudflareaccess.com`. Restart the engine.
5. **CORS preflight ?:** the browser sends an unauthenticated `OPTIONS` preflight to `api.` for the cross-origin POSTs. In the Access application's **Settings ? enable ?Bypass options requests to CORS preflight?** (or add an explicit OPTIONS bypass), else the preflight is redirected to the login page and every API call fails. The engine's CORS middleware (`allow_credentials=false`, no cookies) then answers the preflight.

---

5. The tunnel (engine ? Cloudflare)

```bash
cloudflared tunnel login                                  # browser auth to your zone
cloudflared tunnel create khelsutra-studio                # note the <TUNNEL_UUID>
# copy deploy/cloudflared.config.example.yml ? /etc/cloudflared/config.yml, fill <TUNNEL_UUID>
cloudflared tunnel route dns khelsutra-studio api.khelsutra.guru
cloudflared tunnel run khelsutra-studio                   # or install as a service: `cloudflared service install`
```

```
110     See `deploy/cloudflared.config.example.yml` for the ingress (only `api.khelsutra.guru ? http://127.0.0.1:8000`).
111
112     ---
113
114     ## 6. Verify (the posture test + the CUJ)
115
116     **Exposure posture (must hold ? this is what makes it safe to share):**
117     - From OUTSIDE the box: `curl --max-time 5 http://<box-public-ip>:8000/api/health` ? **refused / timeout** (the engine is `127.0.0.1`-bound; no inbound port). ? The box is not directly reachable.
118     - `curl -i https://api.khelsutra.guru/api/health` **without** a CF session ? a **302 to the Access login** (not a 200). ? The gate is in front.
119     - Open `https://app.khelsutra.guru` in a browser ? CF Access login ? after sign-in, the SPA loads and `/api/health` returns 200 (same session cookie). ?
120
121     **The shareable CUJ (what a tester does):** open `app.khelsutra.guru` ? sign in ? paste a `gs://?` video (or a path under `allowed_video_root`) ? **Process** ? watch progress ? pick rallies ? **Build** ? **download the reel**. All from the page.
122
123     ---
124
125     ## 7. Honest caveats / deferred
126
127     - **`upload_dir` must sit UNDER `allowed_video_root`.** If you enable browser upload, an upload landing outside the jail is rejected by `/api/process` (a functional break, not a hole). Keep `upload_dir` inside the jail (§2.2).
128     - **You MUST set `edge_auth_enabled=true` to expose.** The boot posture (and the in-app JWT verify) only engage then. Do not put the engine behind the tunnel with edge auth off.
129     - **Cross-origin reel download:** the reel streams from `api.` with `Content-Disposition: attachment`, so it downloads even cross-origin ? but verify the filename in your browser during the CUJ (cross-origin `<a download>` filename hints are ignored; the header is the authority).
130     - **Upload size:** Cloudflare's free proxy caps request bodies at **100 MB** ? large matches upload as ?90 MB chunks (the SPA chunked-upload path), or pre-stage to a bucket and ingest the `gs://` URI (already supported).
131     - **Box offline = engine offline.** The Pages SPA stays up and shows a friendly "engine offline" banner (`/api/health` ping); processing resumes when the box is back. Beta removes this with a queue (`HOSTING_PLAN.md` §3).
132     - **Gemini = paid + uploads footage to Google.** Keep the key rotated; use it only for clips you're OK sending.
133
134     ---
135
136     ## 8. Teardown
137     `cloudflared tunnel cleanup khelsutra-studio` + delete the tunnel; remove the Pages custom domain; disable the Access application; stop the engine service. The box's `output/` + `uploads/` retain originals ? sweep them per your retention policy before any wider exposure.
138
139     ---
140     ### Changelog
141     - 2026-06-21 ? created. Split-topology alpha deploy kit (cloudflared + Pages + CF Access) consuming the engine exposure-safety boot posture (engine #294) + the SPA `VITE_API_BASE` (#64). Pairs with `LOCAL_APPLIANCE_ARCHITECTURE.md` (single-origin alternative) + the engine `HOSTING_PLAN.md` §2.
142
```

5. assistant (2026-06-21T19:15:58.956Z)

Now let me verify the load-bearing claims directly against engine code. The two blockers (bind host and gs:// jail) need confirmation. Let me check the engine origin/master.

6. user (2026-06-21T19:16:02.360Z)

679e642 feat(pkg): P2d ? DB upgrade-safety (downgrade guard + backup-before-migrate + export)

(default-OFF) (#307)

7. user (2026-06-21T19:16:02.604Z)

```
from backend.api import auth as edge_auth  # M9: Cloudflare Access edge-auth (default-OFF)

app = FastAPI(title="Sports Highlight Indexer API")


def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
    """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-separated),
    default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
    does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to its
origin."""
    src = os.environ if env is None else env
    raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
    origins = [o.strip() for o in raw.split(",") if o.strip()]
    return origins or ["*"]
```

CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by brow fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so credenti off. The origin list is configurable via the env var above (default ["*"]) so a hosted depl lock it to its origin; the middleware is fixed at startup.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=_cors_origins_from_env(),
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)


def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
    """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
malicious
    web page can point a domain it controls at 127.0.0.1 and script requests to the local
server;
    the browser sends ``Host: attacker.com``, which this rejects (the server only answers
loopback
    Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the default
keeps
    Starlette's TestClient host ``testserver`` so the in-process suite isn't broken. Read from
the
    ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a
routable/edge
    deploy sets its public host (or ``*`` when fronted by an auth gate that validates the
host)."""
    src = os.environ if env is None else env
    raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
    hosts = [h.strip() for h in raw.split(",") if h.strip()]
    # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's
TrustedHostMiddleware
    # matches on host.split(":")[0], which mangles a bracketed IPv6 Host ("[::1]:port"), so IPv6
    # loopback is NOT supported by this guard ? access the appliance via 127.0.0.1/localhost.
    return hosts or ["localhost", "127.0.0.1", "testserver"]
```

DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers a (a remote page can't read responses cross-origin AND its Host header is rejected here

off so an unexpected ``www.`` host is rejected rather than 307-redirected.

```
app.add_middleware(
    TrustedHostMiddleware,
    allowed_hosts=_allowed_hosts_from_env(),
    www_redirect=False,
)


def _resolve_bind_host(config: Optional[AppConfig] = None, cli_host: Optional[str] = None) ->
str:
    """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
``0.0.0.0``
    when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS an auth
gate
    in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a routable
    interface with no auth gate is unsafe, hence the localhost default; widen it consciously."""
    if cli_host and cli_host.strip():
        return cli_host.strip()
    env = os.environ.get("RALLY_BIND_HOST")
    if env and env.strip():
        return env.strip()
    sec = getattr(config, "security", None)
    if getattr(sec, "edge_auth_enabled", False):
        return "0.0.0.0"  # behind a Cloudflare-Access-style edge gate (M9) ? bind all
interfaces
    return "127.0.0.1"
```

--- P2: the friendly `indexer --app` launcher (cross-platform, no native component ? O1

```
def _port_is_free(host: str, port: int) -> bool:
    """True if (host, port) can be bound right now (best-effort; a TOCTOU race is acceptable ?
    uvicorn surfaces a real bind failure loudly)."""
```

8. assistant (2026-06-21T19:16:05.893Z)

The bind-host blocker is confirmed: with `edge_auth_enabled=true` (which §2.2 sets), `_resolve_bind_host` returns `0.0.0.0`, not `127.0.0.1`. The doc's §2.3 comment and §0/§6 "127.0.0.1-bound" framing are false for the prescribed config. Now let me verify the gs:// jail blocker and the §6 posture-test implications.

9. user (2026-06-21T19:16:07.116Z)

```
"""Path-safety helpers for the API boundary (untrusted request fields)."""
import os
from typing import Any, Optional


def output_base(config: Any) -> str:
    """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
    (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``) so the
    chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` / ``output.output_dir``
instead of
    a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior literal.

    This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which may be
a
    ``gs://`` URI you can't ``makedirs``). Accepts the typed AppConfig or the legacy dict-shim
    (``.get("output", {}).get("output_dir")``); any miss or empty value falls back to
``"output"``."""
    try:
        out = config.get("output", {}).get("output_dir")
    except Exception:
```

```
        out = None
    return out or "output"


def safe_output_filename(name: str) -> str:
    """Validate an output filename is a BARE name (no dir part, no traversal, not absolute).

    `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is absolute,
    and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-write
    primitive. Returns the name unchanged when safe; raises ValueError otherwise.
    """
    if not name or not name.strip():
        raise ValueError("output_filename must not be empty")
    if "\x00" in name:
        raise ValueError("output_filename must not contain a null byte")
    if "/" in name or "\\" in name:
        raise ValueError(f"output_filename must not contain a path separator: {name!r}")
    if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
        raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
    return name


def resolve_under_root(path: str, root: str) -> str:
    """Return the real path, asserting it is contained under `root` (raises ValueError if
not)."""
    rp = os.path.realpath(path)
    rr = os.path.realpath(root)
    try:
        contained = os.path.commonpath([rp, rr]) == rr
    except ValueError:  # different drives on Windows
        contained = False
    if not contained:
        raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
    return rp


def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
    """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.

    With no allowed_root (default, single-user local mode) only the null-byte check applies,
    so the local 'point it at any of my videos' workflow is preserved. Setting an
    allowed_root (a future hosted/tenant deployment) enforces containment.
    """
    if "\x00" in path:
        raise ValueError("video_path must not contain a null byte")
    if allowed_root:
        resolve_under_root(path, allowed_root)
```

10. user (2026-06-21T19:16:07.674Z)

```
    """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

    Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
    client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
    runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
    worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
    """
    # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local single-user,
    # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a missing or
    # spoofed identity header fails here with 401, before any work) and tags the job's owner_id.
    try:
        owner_id = edge_auth.resolve_tenant(request.headers, global_config)
    except edge_auth.EdgeAuthError as e:
        raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
```

```
    allowed_root = getattr(getattr(global_config, "security", None), "allowed_video_root", None)
    try:
        validate_input_path(req.video_path, allowed_root=allowed_root)
        # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
        # scheme, which we map to a clean 400 (an unknown scheme must never 500).
        input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config, "storage",
None))
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e)) from e
    # The local-existence fast-fail applies only to a LOCAL input, and stats the RESOLVED local
path
    # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI string. A
    # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified when the
worker
    # fetches it (a fetch miss fails the job loudly). validate_input_path above still runs for
every
    # input: its null-byte guard always applies, and a configured allowed_video_root (hosted
mode)
    # rejects a non-local URI as out-of-root.
    if input_ref.backend == "local" and not os.path.exists(input_ref.key):
        raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
    if req.segmenter and not segmenter_registry.get(req.segmenter):
        available = list(segmenter_registry.list_available().keys())
        raise HTTPException(
            status_code=400,
            detail=f"Segmenter '{req.segmenter}' not found. Available segmenters: {available}"
        )

    job_id = uuid.uuid4().hex
    if not db_instance.create_job(job_id, req.video_path, req.segmenter,
                                  req.player_pool, req.max_frames, owner_id=owner_id,
```

11. assistant (2026-06-21T19:16:12.105Z)

Both blockers confirmed. The critical detail in the engine's own comment (main.py:586): `validate_input_path` runs BEFORE `resolve_input_ref`, and "a configured `allowed_video_root` (hosted mode) rejects a non-local URI as out-of-root." So with the §2.2 config, `gs://` is rejected by the jail ? exactly the verdict's finding. Let me confirm the test that pins this behavior, and check the Host-allowlist startup warning to nail down the §6 posture-test surprise.

12. user (2026-06-21T19:16:13.229Z)

```
def test_process_local_scheme_uri_missing_still_404(env):
    """local://<missing> still 404s ? the existence gate runs on the resolved local key."""
    client = env[0]
    r = client.post("/api/process", json={"video_path": "local:///no/such/ls-missing.mp4"})
    assert r.status_code == 404


def test_process_hosted_mode_rejects_nonlocal_uri(env):
    """Hosted mode (allowed_video_root set) must reject a bucket/Drive URI ? a tenant can't
point
    the engine at an arbitrary bucket. Pins the security contract instead of an incidental side
effect."""
    client, db, tmp_path, mp = env
    root = tmp_path / "tenant"
    root.mkdir()
    mp.setattr(m.global_config.security, "allowed_video_root", str(root))
    for bad in ("gs://b/x.mp4", "s3://b/x.mp4", "gdrive://Sharing/x.mp4"):
        r = client.post("/api/process", json={"video_path": bad})
        assert r.status_code == 400, f"{bad} -> {r.status_code}"
    # A local file UNDER the root is still accepted (passes validation; processing mocked
```

```
elsewhere).
    under = root / "ok.mp4"
    under.write_bytes(b"hashable")
    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
    mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
    r = client.post("/api/process", json={"video_path": str(under), "segmenter": "x"})
    assert r.status_code == 202


def test_cli_nonlocal_input_fetches_and_keeps_uri(tmp_path, monkeypatch):
    """The CLI --video-path block (a second M2 call-site) routes a bucket URI through fetch: the
    segmenter sees a LOCAL path and the DB keeps the source URI."""
    db = Database(str(tmp_path / "cli.db"))
    cfg = AppConfig()
    cfg.output.output_dir = str(tmp_path)
    monkeypatch.setattr(m, "load_app_config", lambda *a, **k: cfg)
    monkeypatch.setattr(m, "Database", lambda *a, **k: db)
    monkeypatch.setattr(m, "global_config", cfg)
    monkeypatch.setattr(m, "db_instance", db)
    monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)

    def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
```

13. user (2026-06-21T19:16:14.040Z)

```
def _exposure_checks(config: Any, env: Mapping[str, str]) -> list[StartupCheck]:
    """Exposure-safety posture for an *exposed* HTTP server. Gated on
``security.edge_auth_enabled``
    (default False ? ``[]`` = strict no-op, today's behaviour). When edge auth is ON ? the "I am
    exposing this engine behind the CF-Access gate" signal ? the gate alone isn't enough: verify
the
    rest of the posture at boot so a half-configured exposure fails fast instead of leaking /
500-ing
    per request. Config-only (no IO beyond the ``find_spec`` auth-extra probe); ``env`` is the
    caller's environment view (so the host-allowlist check is testable)."""
    sec = getattr(config, "security", None)
    if not sec or not getattr(sec, "edge_auth_enabled", False):
        return []  # DEFAULT-OFF: edge auth off ? no posture enforcement (byte-identical to
today).

    out: list[StartupCheck] = []

    # (1) The path-jail is load-bearing once authenticated tenants can hit /api/process: WITHOUT
it,
    # a tenant can point the engine at an ARBITRARY local file (allowed_video_root unset = "any
local
    # file", the single-user default). Fatal for an exposed engine.
    root = getattr(sec, "allowed_video_root", None)
    if not root or not str(root).strip():
        out.append(StartupCheck(
            LEVEL_ERROR, "EXPOSED_NO_VIDEO_ROOT",
            "security.edge_auth_enabled is ON (engine exposed) but security.allowed_video_root
is "
            "unset ? an authenticated tenant could make /api/process read an arbitrary local
file. "
            "Set allowed_video_root to a jail directory before exposing the engine.",
        ))

    # (2) Edge auth cannot verify a token without the team domain + AUD. Today that only fails
on the
    # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate never starts
    # "healthy" then 401s every request.
    team = (getattr(sec, "cf_team_domain", "") or "").strip()
    aud = (getattr(sec, "cf_access_aud", "") or "").strip()
```

```
    if not team or not aud:
        missing = " + ".join(n for n, v in (("cf_team_domain", team), ("cf_access_aud", aud)) if
not v)
        out.append(StartupCheck(
            LEVEL_ERROR, "EXPOSED_EDGE_AUTH_INCOMPLETE",
            f"security.edge_auth_enabled is ON but {missing} is not configured ? the Cf-Access
JWT "
            "cannot be verified (every request would fail). Set security.cf_team_domain "
            "(e.g. https://<team>.cloudflareaccess.com) and security.cf_access_aud (the CF
Access "
            "application AUD tag).",
        ))

    # (3) The auth extra (PyJWT[crypto]) is lazy-imported by auth.verify_cf_access_token. If
it's
    # missing, every request 500s at verify time ? a boot WARN beats a per-request 500. Warning,
not
    # fatal: the auth module's own ImportError is the final authority.
    if not _auth_extra_available():
        out.append(StartupCheck(
            LEVEL_WARN, "EXPOSED_AUTH_EXTRA_MISSING",
            "security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ? Cf-Access "
            "verification will fail at request time. Install the 'auth' extra (pip install -e
.[auth]).",
        ))

    # (4) P0c: an exposed engine binds 0.0.0.0, but the Host-header (DNS-rebinding) guard
defaults to
    # loopback-only when RALLY_ALLOWED_HOSTS is unset (see main._allowed_hosts_from_env) ? it
would
    # then 400 every request the edge forwards with its public Host. Boot WARN beats a silent
400-all.
    if not (env.get("RALLY_ALLOWED_HOSTS", "") or "").strip():
        out.append(StartupCheck(
            LEVEL_WARN, "EXPOSED_HOST_ALLOWLIST_LOOPBACK",
            "security.edge_auth_enabled is ON but RALLY_ALLOWED_HOSTS is unset ? the Host-header
guard "
            "defaults to loopback-only and will 400 forwarded requests. Set RALLY_ALLOWED_HOSTS
to your "
            "public host(s) (or '*' when the edge gate validates the host).",
        ))
```

14. assistant (2026-06-21T19:16:20.494Z)

This is the decisive finding. The startup-posture comment at startup.py:142 confirms: "an exposed engine binds 0.0.0.0" ? the engine itself documents that edge_auth on = 0.0.0.0 bind, directly contradicting the doc's §2.3 "binds 127.0.0.1." And the `EXPOSED_HOST_ALLOWLIST_LOOPBACK` warn proves the cloudflared `Host: api.khelsutra.guru` forwarded requests would 400 unless `RALLY_ALLOWED_HOSTS` is set ? a step the §2.2 config block omits entirely.

Let me confirm the cloudflared service points at 127.0.0.1 (so the bind mismatch is moot for the tunnel path itself but the Host guard still bites), and verify the `gdrive://` scheme is actually recognized by resolve_input_ref (to confirm gs:// would otherwise work without the jail).

15. user (2026-06-21T19:16:21.650Z)

cloudflared tunnel config ? KhelSutra Studio ENGINE (api.khelsutra.guru ? FastAPI or

#

Copy to the engine host (e.g. /etc/cloudflared/config.yml or ~/.cloudflared/config.yml), <TUNNEL_UUID>, then run cloudflared as a service. The tunnel dials OUT to Cloudflar

port is opened on the box ? the engine stays bound to 127.0.0.1:8000 (its safe default) only way in is through Cloudflare Access. See DEPLOY_ALPHA.md.

#

The SPA (app.khelsutra.guru) is NOT served here ? it ships on Cloudflare Pages (DEP This tunnel carries ONLY the engine API.

```
tunnel: <TUNNEL_UUID>
credentials-file: /etc/cloudflared/<TUNNEL_UUID>.json

ingress:
  # The engine FastAPI control plane (POST /api/process, GET /api/jobs, /api/query,
/api/compile,
  # GET /api/highlights, /api/health). Bound to 127.0.0.1:8000 on the box ? never 0.0.0.0.
  - hostname: api.khelsutra.guru
    service: http://127.0.0.1:8000
    originRequest:
      # The async pipeline (fetch ? segment ? stitch) can run for minutes; don't let the tunnel
      # time the request out before the job is submitted (submit returns 202 fast, but be
generous).
      connectTimeout: 30s
      noTLSVerify: false

  # A cloudflared ingress MUST end with a catch-all.
  - service: http_status:404
```

16. user (2026-06-21T19:16:23.527Z)

```
    if args.debug_clip is not None:
        if not args.video_path:
            print("[ERROR] Please provide --video-path to debug a specific clip.")
            return
        run_cli_diagnose_clip(args.video_path, args.debug_clip)
        return

    # Initialize Global Config and DB
    global global_config, db_instance
    global_config = load_app_config()  # returns typed AppConfig
    _enforce_startup_or_exit(global_config)  # M5: fail fast on an incoherent ACTIVE profile

    # The CLI --output-dir overrides the configured output directory. It now lives
    # on the typed model (OutputConfig.output_dir), so every consumer ? including
    # /api/compile ? reads the same value instead of a write-only runtime hack.
    # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
---enforce caller---
115:def _resolve_bind_host(config: Optional[AppConfig] = None, cli_host: Optional[str] = None)
-> str:
845:def _enforce_startup_or_exit(config) -> None:
848:    the exposed server (``include_exposure=True``, M9 hardening), a half-configured exposure
(edge
854:        enforce_startup_checks(config, logger=logger, include_exposure=True)
940:    _enforce_startup_or_exit(global_config)  # M5: fail fast on an incoherent ACTIVE profile
1067:            bind_host = "127.0.0.1"
1068:            port = _resolve_app_port(bind_host, args.port)
1071:            url = f"http://{bind_host}:{port}/"
1080:            bind_host = _resolve_bind_host(global_config, args.host)  # P0c:
config/profile-driven (default 127.0.0.1)
1082:            print(f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{port}...")
1083:            if bind_host not in ("127.0.0.1", "localhost", "::1"):
1084:                print(f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
1092:        uvicorn.run(app, host=bind_host, port=port)
```

17. assistant (2026-06-21T19:16:34.205Z)

I have everything I need. Key reconciliation of the two blockers:

1. **Bind host (blocker):** Confirmed. `--server` branch (main.py:1080) calls `_resolve_bind_host(global_config, ...)` which returns `0.0.0.0` when `edge_auth_enabled=true`. The §2.3 inline comment "binds 127.0.0.1:8000 (do NOT change the bind)" and the cloudflared config comment "the engine stays bound to 127.0.0.1:8000" are both FALSE for the §2.2 config. The mitigating fact: cloudflared dials `127.0.0.1:8000` (outbound only), so the tunnel still works regardless of bind. The real exposure risk is only if an inbound firewall port 8000 is open ? so the §6 posture test `curl http://<box-public-ip>:8000` may NOT refuse at the socket level.

2. **gs:// jail (blocker):** Confirmed by code + test. With `allowed_video_root` set (required to expose), `validate_input_path` runs before scheme resolution and rejects `gs://`/`s3://`/`gdrive://` as out-of-root (400). The §6 CUJ "paste a gs:// video" and §7 "ingest the gs:// URI (already supported)" are mutually exclusive with the exposed config the runbook ships.

3. **New surprise found beyond the verdicts' headline:** The `RALLY_ALLOWED_HOSTS` gap is more severe than "minor." cloudflared forwards `Host: api.khelsutra.guru`; the Host guard defaults loopback-only; so as written, **every tunneled /api request 400s** until `RALLY_ALLOWED_HOSTS` is set ? and the §2.2 config block never tells the owner to set it. The engine only WARNs (non-fatal) at boot, so the engine comes up "healthy" and then 400s everything. This is a functional blocker for the CUJ, distinct from the bind cosmetic issue.

Now I'll assemble the prioritized readiness assessment.

18. user (2026-06-21T19:17:14.727Z)

Structured output provided successfully

19. assistant (2026-06-21T19:17:28.374Z)

Assessment returned. Summary of what I verified against engine `origin/master` (HEAD `679e642`) and khelsutra `master` (`aa8d5e9`):

Overall: needs-doc-fix. The deploy is architecturally sound (the auth/JWT verify, path-jail mechanics, boot posture, CORS, and cloudflared ingress are all verified-correct and well-tested), but if the owner follows DEPLOY_ALPHA.md to the letter, **the CUJ will not work** and two doc claims are factually wrong about the engine's behavior.

Two blockers (both verified in code + tests):
1. The `RALLY_ALLOWED_HOSTS` gap ? §2.2 sets `edge_auth_enabled=true` (which makes the engine bind `0.0.0.0` and default the Host guard to loopback-only), but never tells the owner to set `RALLY_ALLOWED_HOSTS`. cloudflared forwards `Host: api.khelsutra.guru` ? `TrustedHostMiddleware` 400s **every** tunneled request, including the §6 `/api/health` check. The engine only WARNs at boot, so it comes up "healthy" then 400s everything.
2. `gs://` ingest is impossible on the exposed box ? with `allowed_video_root` set (required to expose), `validate_input_path` rejects `gs://`/`s3://`/`gdrive://` as out-of-root (pinned by `tests/test_api_endpoints.py:607`). §6/§7's "paste a gs:// video / already supported" contradicts the §2.2 config it ships.

Notable correction to the verdicts: I treated the `RALLY_ALLOWED_HOSTS` issue as a **blocker**, not the "minor" the §2.3 verdict folded it into ? it breaks the whole tunneled flow, distinct from the (cosmetic) 127.0.0.1-vs-0.0.0.0 bind claim. The cleanest single fix is adding `export RALLY_BIND_HOST=127.0.0.1` to §2.2, which both restores the doc's claimed loopback bind and sidesteps the Host-guard 400s (cloudflared dials 127.0.0.1 either way).

Key files: `C:\Users\avidu\Projects\khelsutra\deploy\DEPLOY_ALPHA.md`, engine `backend/main.py:115-128` (bind), `backend/main.py:575-583` (jail-before-scheme), `backend/config/startup.py:142-147` (Host-allowlist warn), `tests/test_api_endpoints.py:607`.